

UNIVERSITÉ HASSAN II
École Normale Supérieure de l'Enseignement Technique
Département : Mathématique Informatique

RAPPORT TD

Boucles et fonctions

Nom et prénom : Imrane ANOUNOU

Filière : Génie Logiciel (GLSID)

Encadrant : M.MAWANE

Année universitaire : 2026–2027

TD code en github : <https://github.com/IMRANEANOUNOU/TD-BOUCLE-FONCTIONS.git>

Mohammedia, 02 Octobre 2026

1 Exercice 1 —

Question 1

programme 1 : 0 1 2 3 4
programme 1 : 0 1 2 3 4
programme 3 : 1 2 3 4 5

Solution Python

```
print("Programme 1")
for k in range(0, 5):
    print(k)

print("Programme 2")
for k in range(5):
    print(k)

print("Programme 3")
for k in range(1, 6):
    print(k)
```

Sortie

```
programme 1 :
0 1 2 3 4
programme 2 :
0 1 2 3 4
programme 3 :
1 2 3 4 5
```

Question 2

les valeurs finales de k et de a sont $k = 4$ et $a = 16$

Solution Python

```
a = 1
for k in range(1, 5):
    a *= 2

print(f"a = {a} et k = {k}")
```

Sortie

```
a = 16 k=4
```

2 Exercice 2 —

Question 1

On calcule la somme

$$u = 0^3 + 1^3 + 2^3 + \cdots + n^3.$$

Question 2

on va supprimer ligne "print(x)"

3 Exercice 3 —

Solution Python

```
n = int(input("Entrer un entier naturel : "))
x = float(input("Entrer un reel : "))

res = 0
for i in range(n + 1):
    res += i * x**i

print("Le resultat est :", res)
```

Sortie

```
Entrer un entier : 2
Entrer un reel : 2
0
2
10
resultat est : 10
```

4 Exercice 4 — Moyenne de notes

Le programme demande le nombre de notes, vérifie que chaque note appartient à l'intervalle $[0, 20]$, puis calcule la moyenne des notes valides.

Solution Python

```
n = int(input("Donner le nombre de notes : "))
somme = 0

if n <= 0:
    print("Le nombre de notes doit etre positif.")
else:
    notes_valides = True
    for i in range(n):
        note = float(input(f"Note {i + 1} : "))
        if 0 <= note <= 20:
            somme += note
        else:
            print("Entrée invalide ! Une note doit être entre 0 et 20.")
            notes_valides = False
            break

    if notes_valides:
        print("La moyenne est :", somme / n)
```

Sortie

```
python exercice4.py
Donner le nombre de notes : 3
Note 1 : 10
Note 2 : 20
Note 3 : 11
La moyenne est : 13.666666666666666
```

5 Exercice 5 — Fonction factorielle

La fonction suivante renvoie $n!$, avec $0! = 1$ et, pour $n \geq 1$, $n! = n(n-1)!$.

Solution Python

```
def factorielle(n):
    if n == 0:
        return 1
    return n * factorielle(n - 1)

x = int(input("Entrer un entier naturel : "))
if x < 0:
    print("La factorielle n'est définie ici que pour n >= 0.")
else:
    print(f"{x}! = {factorielle(x)}")
```

Sortie

```
admin@linux competence numerique/TPs/TP1 python exercice5.py
Entrer un entier naturel : 3
3! = 6
```

6 Exercice 6 —

Question 1

valeur = -1

Solution Python

```
valeur = 5
while valeur >= 0:
    valeur -= 1
print(valeur)
```

Sortie

-1

Question 2

Prévoir le résultat des quatre variantes proposées dans le support. Les quatre programmes sont reproduits ci-dessous.

Solution Python

```
# Programme 1
a = 0
while a <= 10:
    a = a + 2
print(a)
```

```
# Programme 2
a = 10
while a >= 0:
    a = a - 2
print(a)

# Programme 3
a = 0
while a <= 10:
    print(a)
    a = a + 2

# Programme 4
a = 0
while a <= 10:
    a = a + 2
```

Sortie

```
#programme 1 : 12
#programme 2 : boucle infinie
#programme 3 : 0 2 4 6 8 10
#programme 4 : 2 4 6 8 10 12
```

7 Exercice 7 —

Solution Python

```
def prog1(a, n):
    s = 1
    k = 1
    while k <= n:
        k += 1
        s *= a
    return s

a = float(input("Entrer a : "))
n = int(input("Entrer n : "))
print("a^n =", prog1(a, n))
```

Sortie

```
c'est a a la puissance n : a^n
exemple :
Entrer a : 2
Entrer n : 2
a^n = 4.0
```

8 Exercice 8 — Somme harmonique

Question 1+2

Solution Python

```
def somme_harmonique(n):
    somme = 0
    for k in range(1, n + 1):
        somme += 1 / k
    return somme

somme = 0
n = 0
while somme < 8:
    n += 1
    somme += 1 / n

print("Le plus petit n est :", n)
print("H_n =", somme)
```

Sortie

Le plus petit n est : 1674
 $H_n = 8.000485571995782$

]Exercice 9 —

Question 1–2

Solution Python

```
from random import *
def game() :
    n = randint(0,1000)
    user_input = int(input("Entrer un nombre entre 1 et 1000 : "))
    max_essai = 1
    while n!=user_input and max_essai <=8 :
        if user_input > n:
            print("Le nombre rechercher est plus petit")
        else :
            print("Le nombre rechercher est plus grand")
        max_essai=max_essai+1
        user_input = int(input("nouvel essai : "))

    if(user_input == n):
        print("Bravo ! Vous avez trouve la bonne reponse")
        print(f"nombre d'essais : {max_essai}")
    else :
        print("Vous avez perdu , le nombre est : ",n)

game()
```

Sortie

*Entrer un nombre entre 1 et 1000 : 200
Le nombre rechercher est plus grand
nouvel essai : 260
Le nombre rechercher est plus grand
nouvel essai : 266
Bravo ! Vous avez trouve la bonne reponse
nombre d'essais : 3*

]Exercice 10 —

Solution Python

```
def factorielle(n):  
    if n == 0:  
        return 1  
    return n * factorielle(n - 1)  
  
def approximation_e(epsilon):  
    n = 0  
    u = 1.0  
  
    while 3 / factorielle(n) >= epsilon:  
        n += 1  
        u += 1 / factorielle(n)  
  
    return u, n  
  
epsilon = float(input("Entrer epsilon (> 0) : "))  
if epsilon <= 0:  
    print("epsilon doit être strictement positif.")  
else:  
    valeur, n = approximation_e(epsilon)  
    print("Approximation de e :", valeur)  
    print("Indice utilisé :", n)
```

Sortie

*Entrer epsilon (> 0) : 0.0001
Approximation de e : 2.71827876984127
Indice utilisé : 8*

]Exercice 11 —

Solution Python

```
def terme(n):  
    u = 1.0  
    for i in range(n):  
        u = u / (u + 1)  
    return u  
  
n = int(input("Entrer n : "))  
if n < 0:  
    print("n doit être naturel.")  
else:  
    print(f"u_{n} = {terme(n)}")
```

Sortie

Entrer n : 3
u₃ = 0.25

Conjecture. Les premières valeurs suggèrent la formule

$$u_n = \frac{1}{n+1}.$$

]Exercice 12 —

Question 2

Solution Python

```
def fibonacci(n):  
    if n < 2:  
        return 1  
  
    a = 1  
    b = 1  
    for i in range(2, n + 1):  
        a, b = b, a + b  
    return b  
  
def calcul_v(n):  
    if n < 1:  
        raise ValueError("v_n nécessite n >= 1")  
    return fibonacci(n + 1) * fibonacci(n - 1) - fibonacci(n)**2  
  
n = int(input("Entrer n (n >= 1) : "))  
if n < 1:  
    print("n doit être supérieur ou égal à 1.")  
else:  
    print(f"u_{n} =", fibonacci(n))  
    print(f"v_{n} =", calcul_v(n))
```


Sortie

Entrer n ($n \geq 1$) : 3

$$u_3 = 3$$

$$v_3 = 1$$

Conjecture. Les calculs donnent une alternance de valeurs $v_n = (-1)^{n+1}$ pour $n \geq 1$ avec la convention $u_0 = u_1 = 1$.